JavaScript: Array

Basic Understanding

- Arrays are variables. But array can contain many values (called elements).
- We use position number (called index) to access elements of an array.
  - Oppose to object that use name (called key) to access elements
- In JavaScript array is an object.

Array properties and methods you need to know #1

- `array.length` property
- `array.push(el)`
- `array.pop()`
- `array.shift()`
- `array.unshift(el)`
- `Array.isArray(arr)`

Accessing array's element

```
const cars = ["Toyota", "Honda", "BMW", "Tesla", "Lexus"];

// Index of the first elelment is 0 (zero)
console.log(cars[0]);
console.log(cars[1]);
const index = 0;
console.log(cars[index]);

//Trying to access element that does not exist
console.log(cars[9]);

// Using arr.length property
console.log(cars.length);
console.log(cars[cars.length]);
console.log(cars[cars.length-1]);
```

`push` and `pop`

```
const cars = ["Toyota", "Honda", "BMW", "Tesla", "Lexus"];

console.log(cars)

cars.push("Suzuki")
console.log(cars)

cars.pop()
console.log(cars)

cars.pop()
console.log(cars)
```

`shift()` and `unshift()`

```
const cars = ["Toyota", "Honda", "BMW", "Tesla", "Lexus"];

console.log(cars)
const selectedCar = cars.shift()

console.log(cars)
console.log(selectedCar)

cars.unshift("Lambo")
console.log(cars)
```

`typeof` and `Array.isArray()`

```
const cars = ["Honda", "BMW"];
const user = { name: "John Ive", age: 33 }
const isAlive = true;
const msg = "Running..."

console.log(typeof cars)
console.log(typeof user)
```

```
console.log(typeof isAlive)
console.log(typeof msg)
console.log("----")
console.log(Array.isArray(cars))
console.log(Array.isArray(user))
```

Looping through arrays

```
const cars = ["Toyota", "Honda", "BMW", "Tesla", "Lexus"];
console.log("CARS:", cars);
// console.log("LENGTH:", cars.length)

/*
for(let i=0; i<5; i++) {
    console.log(i);
}
*/

/*
for(let i=0; i<cars.length; i++){
  console.log(i);
  // console.log(cars[i])
  // console.log(i, "is" , cars[i]);
}
*/

/*
cars.forEach((el, idx, arr) =>{
  console.log(el, idx, arr);
});
*/
```

Array Destructuring

```
const cars = ["Toyota", "Honda", "BMW", "Tesla", "Lexus"];
[car1, car2] = cars;
```

```
// [, car2] = cars;
// [,, car3] = cars;

console.log(car1, car2);
```

Array methods you need to know #2

- `arr1.concat(arr2, arr3, arr4)`
- `arr.join(seperator)`
- `arr.include(el, start)`
- `arr.slice(start, end)`

```
const fruits = ["Banana", "Apple"]
const fruits2 = ["Orange"]
const fruits3 = ["Pineapple", "Mango"]

const allFruits = fruits.concat(fruits2, fruits3)
console.log(allFruits)

console.log(allFruits.join(" | "))



console.log(allFruits.includes("Banana"))
console.log(allFruits.includes("banana"))
console.log(allFruits.includes("Banana", 1))

let newFruits = allFruits.slice(2)
console.log(allFruits)
console.log(newFruits)

newFruits = allFruits.slice(2, allFruits.length)
console.log(allFruits)
console.log(newFruits)
```

Array map

```
const numbers = [10, 12, 5, 25, 30];

// array.map() visit every elements with your provided function
const myFunction = (el, idx, arr, dummy) => {
    // notice what being passed as function's parameters
    console.log(el, idx, arr, dummy);
}
numbers.map(myFunction);

// passing an anonymous arrow function
numbers.map(el => console.log(el))

// array.map() return a new array
const newNumbers = numbers.map(el => { return el+1000 })
console.log(newNumbers)
console.log(numbers)    // orignal array

// array.map() work very well with arrow function
const newNumbers2 = numbers.map(el => el+1000)
console.log(newNumbers2)
```

Challenge:

From this array...

```
[ 10000, 12000, 5000, 25000, 30000 ]
```

convert to...

**
10,000
**

**
12,000
**

**
5,000
**

**
25,000
**

**
30,000
**

Hint:

```
const total = 2500;
console.log(total.toLocaleString('th-TH'))
```

Solution:

```
const numbers = [10, 12, 5, 25, 30];

console.log(numbers)
console.log(numbers.map(n => n+1000))
console.log(numbers.map(n => n*1000))
const total = 2500;
console.log(total.toLocaleString('th-TH'))
console.log(numbers.map(n => `<li>${(n*1000).toLocaleString('en-US')}</li>`).join("\n'
```

Array filter

```
const numbers = [1,2,3,4,5,6,7];

// array.filter() visit every elements with your provided function
numbers.filter((el,i,a,dummy) => {
    // console.log(el,i,a, dummy)
})

const greaterThanFour = numbers.filter(e => e>4)
console.log(greaterThanFour)
```

```
const students = [
    {id: 34223, name:"Adam", score: 54},
    {id: 35121, name: "Freda", score: 65},
    {id: 34141, name: "Molly", score: 32}
    ]
```

```
const qualifiedStudents = students.filter(el => el.score >= 50)
console.log(qualifiedStudents)
```

List all student that have letter m in his/her name

```
const haveM = students.filter(student => student.name.toLowerCase().includes("m"))
console.log(haveM)
```

Array find

```
const numbers = [1,2,3,4,5,6,7]

// array.find() stop when criteria meet
const foundTarget = numbers.find(el => {
    console.log(el)
    return el >= 4;
})
console.log("foundTarget",foundTarget)

const foundNumber = numbers.find(el => el > 4)
console.log("foundNumber", foundNumber)
```

Note: find() return single element

Array reduce

```
//    arr.reduce(function(accumulate, currentValue, curIndex, arr) {}, initialValue)

const numbers = [2,4,8,16]
const result = numbers.reduce((acc,el,idx,arr) => {
    console.log({acc,el,idx,arr})
    // return acc + el
    // return acc + el/arr.length
}, 0)
```

```
console.log(result)
```

```
const numbers = [2,4,8,16]
const result = numbers.reduce((acc,el,idx,arr) => {
    console.log({acc,el,idx,arr})
    /*
    const sum = acc.sum + el
    const avg = acc.avg + el/arr.length
    const min = acc.min > el ? el : acc.min
    const max = acc.max > el ? acc.max : el
    return {sum,avg,min,max}
    */
}, {sum:0, avg:0, min:0, max:0})

console.log(result)
```